MM-41

A modular programmable RPN calculator for real, complex and decimal scalars, strings and matrices

User Manual, version 1.2

Mico Mrkaic

November 2026

MM-41 is free software under the GNU General Public License, version 3 or later.
Source, issues and releases: `https://github.com/micomrkaic/MM-41`.

Contents

1 Introduction **4**
1.1 Notation used in this manual 4

2 Getting started **5**
2.1 Building and installing 5
2.2 A first session 5
2.3 Getting help inside the calculator 5

3 Entering data **6**
3.1 Numbers 6
3.2 Complex numbers 6
3.3 Strings 6
3.4 Matrices 7
3.5 Comments 7
3.6 Execution tokens 7
3.7 Shell commands 7

4 The stack **7**
4.1 Display 8

5 Arithmetic and functions **8**
5.1 Basic arithmetic . . . 8
5.2 Exponentials, logarithms, trigonometry . . . 9
5.3 The promotion rule . . . 9
5.4 Special functions and the normal distribution . . . 9
5.5 Comparisons and logic . . . 9

6 Decimal numbers **10**
6.1 Writing decimals . . . 10
6.2 Decimal absorbs binary . . . 10
6.3 Display . . . 11

7 Matrices **11**
7.1 Creating matrices . . . 11
7.2 Arithmetic . . . 11
7.3 Linear algebra . . . 12
7.4 Elements and shape . . . 12
7.5 Reductions and statistics . . . 13
7.6 Polynomials . . . 13

8 Strings **13**

9 Registers **14**

10 Words: programming the calculator **14**
10.1 Conditionals . . . 14
10.2 Loops . . . 15
10.3 Saving words . . . 15
10.4 Words from modules . . . 15

11 Numerical methods **16**

12 Dates, time and the sun **17**
12.1 Workdays and holidays . . . 17

13 Finance **19**
13.1 Time value of money . . . 19
13.2 Amortisation . . . 19
13.3 Cash flows: NPV and IRR . . . 20
13.4 Interest-rate conversions and depreciation . . . 20
13.5 Bonds . . . 21
13.6 Dates in the finance module . . . 21
13.7 Dated cash flows: XNPV and XIRR . . . 22

14 Programs **22**

15 Modules **23**
15.1 Built-in modules 23
15.2 Word modules 23
15.3 Compiled modules 24
15.4 Which kind to write 25

16 Batch mode and scripting **25**

17 Configuration and files **26**

18 Errors **26**

A Command reference **28**

B The std word module **40**

C Coming from MM-15, MM-14, MM-11 or MM-B1 **41**

1 Introduction

MM-41 is a stack-based calculator in the tradition of the HP-41C and of FORTH, written in C17 on top of the GNU Scientific Library and GNU Readline. You type numbers and commands; numbers go on a stack and commands take their arguments from it. Beyond the usual scalar functions it handles complex numbers, exact decimals for money, strings, and real and complex matrices, and it is programmable in three ways: user-defined *words* with conditionals and loops, numerical methods (integration, root finding, minimisation, differentiation) that take any word as the function, and HP-41 style line programs with labels and jumps.

Like the HP-41, it is built from *modules*. The core knows about numbers, the stack, words and errors; everything else—matrices, numerical methods, dates, strings, finance—is a module, and you can add your own, either as a file of word definitions or as compiled code (Section 15). MM-41 unifies the earlier MM-11, MM-14, MM-15 and MM-B1 calculators.

The design has a few fixed points worth knowing before you start.

- **Everything is a token on a line.** Whitespace separates tokens; a line is evaluated left to right. There is no operator precedence because there are no infix operators.
- **Every token is atomic.** If a command fails—wrong type, singular matrix, division by zero—the stack is put back exactly as it was before that token, the error is printed, and the rest of the line is skipped. You never end up with half a result.
- **Two promotion rules.** A real function applied outside its real domain returns a complex number rather than `nan`: `-1 sqrt` is $(0, 1)$, `2 asin` is $(1.5708, 1.3170)$, and a real matrix with one such entry becomes a complex matrix. And decimal absorbs binary: a decimal combined with a real gives a decimal, so `0.1d 0.2 +` is exactly `0.3`.
- **Instant startup, no ceremony.** The calculator starts in a few milliseconds, reads your saved words and configuration from `~/.config/mm41`, and is equally at home in a terminal session and in a shell pipeline.

1.1 Notation used in this manual

Commands are shown in **`bold monospace`**. Their effect on the stack is described by a *stack picture*, written deepest-first with the top of the stack at the right:

```
y x -- y+x
```

means that the command consumes two items—`y` below `x`—and leaves their sum. A picture that lists the same name on both sides (`A -- A rows cols`) means the item stays on the stack.

Transcripts show what you type, followed by the calculator's output. In the interactive session the whole stack is printed after every line; in the transcripts we usually show only the relevant part. Lines beginning with `\` are comments and are ignored by the calculator; we use them to annotate results.

```
2 3 +            \ 5
(1,2) (3,-1) *   \ (5,5)
```

2 Getting started

2.1 Building and installing

MM-41 needs a C17 compiler, GNU make, GNU Readline and the GNU Scientific Library (GSL). On Debian or Ubuntu:

```
sudo apt install build-essential libreadline-dev libgsl-dev
```

and on macOS with Homebrew, `brew install gsl readline`. Then:

```
git clone https://github.com/micomrkaic/MM-41
cd MM-41
make              # builds bin/release/mm41
make test         # 21 unit cases + golden files, run under AddressSanitizer
make install      # into ~/.local: bin/mm41, the man page, this manual, and docs
```

`make install` also seeds `~/.config/mm41/` with the default configuration (kept if you already have one) and the shipped word modules (always replaced, so they match the program). `PREFIX=/usr/local make install` installs system-wide; `make uninstall` removes exactly what was installed. Make sure `~/.local/bin` is on your `PATH`.

2.2 A first session

```
$ mm41
MM_RPN>> 2 3 +
[0] ℝ : 5.000000
MM_RPN>> 4 *
[0] ℝ : 20.000000
MM_RPN>> dup sqrt
[0] ℝ : 20.000000
[1] ℝ : 4.472136
MM_RPN>> q
```

The prompt is `MM_RPN>>`. After each line the whole stack is printed, one item per line, deepest first: the index in brackets, a type glyph (ℝ real, ℂ complex, $\mathcal{S}$ string, M$\mathbb{R}$ and M$\mathbb{C}$ real and complex matrices), and the value. Position `[0]` is the *bottom* of the stack; the highest index is the top, which is where commands take their arguments.

The REPL has GNU Readline editing: arrow keys recall history (kept in `.rpn_history` between sessions), and TAB completes command and word names. **`q`** exits, saving the configuration; **`undo`** restores the stack as it was before the previous line.

2.3 Getting help inside the calculator

`help`
: a one-screen summary of the input syntax and the command families.

`listfcns`
: every command, grouped by module and category, with its stack picture.

`modules`
: the loaded modules with their versions.

"name" usage
the stack picture, description and an example for one command—or, for a user word, its definition:

```
MM_RPN>> "mod" usage
"mod" usage: y x -- y mod x   [Arithmetic]
    floored modulo (result has the sign of x)
    Example: -7 3 mod
```

Outside the calculator there is `man mm41`, and `mm41 --commands` prints the complete reference as Markdown (it is also Appendix A of this manual).

3 Entering data

All input is case sensitive. A line is split into tokens at whitespace; the exceptions are strings, complex literals and matrix literals, which may contain spaces.

3.1 Numbers

Real numbers in the usual forms: `3`, `-1.25`, `.5`, `1e3`, `-2.5e-3`. A leading `+` is accepted. Numbers are IEEE double precision; **`inf`** and **`nan`** push the special values, and **`pi`**, **`e`** and **`gravity`** ($9.80665\,\mathrm{m/s^2}$) push constants. A number with the suffix `d` (`0.1d`, `1250.00d`) is a *decimal*, an exact 34-digit number for money; Section 6 explains decimals and the **`decimal`** mode in which every literal is one.

3.2 Complex numbers

A complex number is written in parentheses, real part first: `(1,2)`, `(-1.2e-4, 0.7e2)`. Spaces inside the parentheses are allowed. A complex number prints as `(1.000000, 2.000000i)`. The commands **`re`**, **`im`**, **`abs`**, **`arg`**, **`conj`**, **`re2c`**, **`split_c`** and **`j2r`** convert between the representations:

```
(3,4) abs        \ 5
(3,4) arg        \ 0.927295  (radians)
(1,2) split_c    \ 1 2       (two reals)
1 2 j2r          \ (1,2)     (imaginary part on top)
5 re2c           \ (5,0)
```

Reals and complex numbers mix freely in arithmetic; the result is complex whenever an operand is.

3.3 Strings

Strings are double-quoted: `"hello world"`. The escapes `\"`, `\\`, `\n` and `\t` are recognised. Strings are used for dates, file names, word names (`"sq" usage`), and as input to **`eval`**; see Section 8.

3.4 Matrices

An inline matrix gives the dimensions, a dollar sign, and the elements in row-major order:

```
[2 2 $ 1 2 3 4]            \ the 2x2 matrix  1 2 / 3 4
[1 3 $ 1 2 3]              \ a row vector
[3 1 $ 1 2 3]              \ a column vector
[2 2 $ 1 (0,1) 0 1]        \ one complex entry makes a complex matrix
```

A matrix can also be read from a whitespace-separated text file, giving its shape:

```
[2, 2, "m.txt"]             \ reads four numbers from m.txt, row by row
```

The stack shows a matrix as `MℝX: 2 x 2 matrix`; **pm** (or **print**) prints it in full. Section 7 covers what you can do with them.

3.5 Comments

\ and # start a comment that runs to the end of the line. This works in the REPL, in scripts, in word definitions and in program files.

3.6 Execution tokens

`@name` does not run `name`; it pushes a reference to it, an *execution token*, as a real number. The numerical methods (Section 11) and **exec** consume these: `0 pi @sin integrate` integrates the sine over $[0, \pi]$. Any command or word, from any module, can be referenced this way.

3.7 Shell commands

A line that starts with `!` is run by the shell and its standard output is pushed as a string:

```
!date +%Y
```

4 The stack

The stack holds up to 100 items of any type. The classic FORTH words manipulate it:

dup	`x -- x x`	duplicate the top
drop	`x --`	discard the top
swap	`a b -- b a`	exchange the top two
over	`a b -- a b a`	copy the second item to the top
nip	`a b -- b`	discard the second item
tuck	`a b -- b a b`	copy the top below the second item
pick	`... n -- ... v`	copy the item at depth n to the top (`0 pick` is **dup**, `1 pick` is **over**)
roll	`... n -- ...`	move the item at depth n to the top (`1 roll` is **swap**, `2 roll` is FORTH's `rot`)
depth	`-- n`	number of items on the stack
clst	`... --`	clear the stack

```
10 20 30 2 roll     \ 20 30 10
10 20 30 1 pick     \ 10 20 30 20
```

savestack writes the whole stack to `stack_backup.dat` in the configuration directory and **loadstack** pushes it back—handy for carrying a set of matrices from one session to the next.

4.1 Display

print prints the top item (a matrix in full), **ps** prints the whole stack, **pm** prints the matrix on top in full. All three leave the stack unchanged. A matrix prints with four decimals (scientific notation for very large or small entries), except that a matrix whose entries are all whole numbers—dates, counts, an integer cash-flow table—prints them as integers.

Numbers are shown in fixed-point with six decimals by default. **setprec** sets the number of decimals (0–20), and **sfs** toggles between fixed-point and scientific notation, where the precision is the number of significant digits:

```
10 setprec 1 3 /      \ 0.3333333333
sfs 1 3 /             \ 0.3333333333  (6 significant digits would be 0.333333)
3 setprec 12345.678   \ 1.23e+04
sfs                   \ back to fixed point
```

Both settings are saved in `config.txt` when you quit.

5 Arithmetic and functions

5.1 Basic arithmetic

+, **-**, *****, **/** and **^** (also spelled **pow**) work on reals, complex numbers and matrices (Section 7 explains the matrix cases). Division by zero is an error, not `inf`.

chs	`x -- -x`	change sign
inv	`x -- 1/x`	reciprocal
abs	`z -- |z|`	absolute value or complex magnitude
sqrt	`x -- sqrt(x)`	square root
intg	`x -- int(x)`	integer part, truncated toward zero (`-3.7 intg` is -3)
frac	`x -- frac(x)`	fractional part, $x - \mathrm{intg}(x)$ (`-3.7 frac` is -0.7)
mod	`y x -- y mod x`	floored modulo; the result has the sign of x (`-7 3 mod` is 2)
fact	`x -- x!`	factorial, $\Gamma(x+1)$ for non-integers
ncr	`n k -- C(n,k)`	combinations
npr	`n k -- P(n,k)`	permutations

The three percentage functions follow the HP convention of *keeping* the base value y so that it is available for the next step:

```
200 25 %        \ 200 50      25% of 200
200 50 %t       \ 200 25      50 is 25% of 200
80 100 %chg     \ 80 25       from 80 to 100 is +25%
```

5.2 Exponentials, logarithms, trigonometry

exp, **ln**, **log** (base 10); **sin**, **cos**, **tan**, **asin**, **acos**, **atan**; **sinh**, **cosh**, **tanh**, **asinh**, **acosh**, **atanh**. Angles are in radians. All of them accept real and complex arguments and apply elementwise to matrices.

5.3 The promotion rule

Real functions whose result would not be real return a complex number. The rule is applied consistently, so a calculation never produces `nan` where a complex answer exists:

Command	Promotes when	Example
sqrt	$x < 0$	`-4 sqrt` $\to (0, 2)$
ln, **log**	$x < 0$	`-1 ln` $\to (0, \pi)$
asin, **acos**	$\lvert x\rvert > 1$	`2 asin` $\to (1.5708, 1.3170)$
acosh	$x < 1$	`0.5 acosh` $\to (0, 1.0472)$
atanh	$\lvert x\rvert > 1$	`2 atanh` $\to (0.5493, 1.5708)$
^	$y < 0$ and x not an integer	`-8 3 inv ^` $\to (1, 1.7321)$

A real matrix with one such entry becomes a complex matrix: `[2 2 $ 1 -2 3 4] ln` is complex. The limits $\pm\infty$ remain real where they are the correct answer (`0 ln` is `-inf`).

5.4 Special functions and the normal distribution

gamma, **ln_gamma**, **beta** and **ln_beta** wrap the GSL special functions. **npdf**, **ncdf** and **nquant** are the standard normal density, distribution function and quantile:

```
1.96 ncdf        \ 0.975002
0.975 nquant     \ 1.959964
5 gamma          \ 24
2 3 beta         \ 0.083333
```

5.5 Comparisons and logic

eq, **neq**, **lt**, **leq**, **gt**, **geq** leave a flag: 1 for true, 0 for false. **and**, **or** and **not** treat 0 as false and anything else as true. Complex numbers compare by magnitude; strings support **eq** and **neq**; matrices compare elementwise and produce a matrix of flags, and a scalar compared with a matrix is compared with every element.

```
2 3 lt                   \ 1
"abc" "abc" eq           \ 1
[1 3 $ 1 5 3] 2 gt pm    \ | 0 1 1 |
```

Flags are what the control words inside user words test (Section 10).

6 Decimal numbers

Binary floating point cannot represent one tenth, so `0.1 0.2 +` is `0.30000000000000004` on every computer. For most of science that is irrelevant; for money it is not. MM-41 therefore has a fifth stack type, the *decimal*: a 34-digit decimal floating-point number with round-half-even arithmetic, the same arithmetic as SQL `NUMERIC`, Java `BigDecimal` and Python's `decimal` module, implemented by IBM's decNumber library. A decimal shows on the stack with the glyph 𝔻:

```
0.1d 0.2d +            \ [0] 𝔻 : 0.300000
0.1 0.2 + 0.3 eq       \ 0   (binary)
0.1d 0.2d + 0.3d eq    \ 1   (decimal)
```

6.1 Writing decimals

A number with the suffix `d` is a decimal literal; `b` forces a binary one. In *decimal mode*, entered with **`decimal`** and left with **`binary`**, every number without a suffix is a decimal, and the prompt becomes `MM-41[dec]>>`. The mode is saved in `config.txt` as `number_mode`, so a calculator used for bookkeeping can start in decimal mode every time.

```
decimal
19.99 3 * 0.0825 1 + *         \ 64.917525   (exact: 59.97 x 1.0825)
2 round                        \ 64.92
binary
```

`>dec` converts a real to a decimal by its *shortest round-trip representation*, so `0.1 >dec` is exactly `0.1` and not `0.1000000000000000055511151231257827`; **`>bin`** goes the other way and keeps 16 or 17 of the 34 digits. **`round`** rounds a number to n decimal places, half-even, and always leaves a decimal: `2.345d 2 round` is 2.34 and `2.355d 2 round` is 2.36. **`isdec`** and **`decmode`** test the type of the top item and the current mode.

6.2 Decimal absorbs binary

A decimal combined with a real gives a decimal (the real is converted first, as with **`>dec`**): `1d 3 /` is `0.333333333333333333333333333333333333`. The operators and functions defined for decimals are

arithmetic	**`+ - * / ^ chs abs inv mod`**
functions	**`sqrt exp ln log intg frac round`**
percentages	**`% %t %chg`**
comparisons	**`eq neq lt leq gt geq and or not`**
elsewhere	**`sto`** and **`rcl`**, **`savestack`** and **`loadstack`**, **`int2str`**, the control words, and the finance module (Section 13), which computes in decimals throughout

Everything else—trigonometry, complex numbers, matrices, the numerical methods—is binary only, and a decimal *value* given to them is an error that tells you to convert with **`>bin`**. A decimal given where a command wants an index, a count, a dimension, a precision or a flag is simply converted, since nothing can be lost: `2 setprec`, `1 pick`, `5 rcl`, `3 3 eye`, `0 pi @sin integrate`

and `3 0 do ... loop` all work in decimal mode. This is deliberate: the one direction in which digits are lost should be visible in the transcript. A decimal that would need a complex result (`-4d sqrt`, `-1d ln`) is likewise an error rather than a silent promotion, because money has no imaginary part. The numerical methods accept a word that returns a decimal and convert the result, so `: f dup * 2d - ; 0 3 @f fzero` works.

6.3 Display

In fixed-point display a decimal is shown rounded half-even to the **`setprec`** number of decimals, the natural way to show money; in scientific display (**`sfs`**) it is shown *as stored*, with all its digits and its exponent, which is how you inspect it:

```
1d 3d /                 \ [0] 𝔻 : 0.333333
sfs                     \ [0] 𝔻 : 0.33333333333333333333333333333333333
1e-7d                   \ [1] 𝔻 : 1E-7
```

Decimals keep their trailing zeros, as the General Decimal Arithmetic specification prescribes: `1.10d 2d *` is `2.20`.

7 Matrices

7.1 Creating matrices

Besides the literal forms of Section 3, there are constructors:

```
3 eye                       \ 3x3 identity
2 3 ones                    \ 2x3 matrix of ones
2 3 zeroes                  \ 2x3 matrix of zeros
2 2 rand                    \ uniform(0,1) entries
2 2 randn                   \ standard normal entries
0 0.25 1 rrange             \ row vector 0 0.25 0.5 0.75 1
[1 3 $ 1 2 3] to_diag       \ 3x3 diagonal matrix with 1 2 3 on the diagonal
```

7.2 Arithmetic

- **`+`** and **`-`** add and subtract matrices of the same shape. A scalar is added to every element.
- **`*`** is the matrix product; a scalar scales every element.
- **`/`** divides a matrix by a scalar elementwise; `x A /` is x/a_{ij} elementwise; and `A B /` is AB^{-1} (right division).
- **`^`** raises a square matrix to a non-negative integer power (A^0 is the identity).
- **`.*`**, **`./`**, **`.^`** are the elementwise product, quotient and power; either operand may be a scalar.
- A real matrix combined with a complex matrix or complex scalar is promoted to complex first.

```
[2 2 $ 1 2 3 4] dup * pm        \ |  7 10 |  matrix product
                                \ | 15 22 |
[2 2 $ 1 2 3 4] dup .* pm       \ |  1  4 |  elementwise
                                \ |  9 16 |
```

```
[2 2 $ 1 2 3 4] 10 + pm        \ | 11 12 | / | 13 14 |
[2 2 $ 1 2 3 4] 3 ^ pm         \ | 37 54 | / | 81 118 |
```

The unary functions—**sqrt**, **exp**, **ln**, the trigonometric and hyperbolic functions, **abs**, **chs**, **inv**, **intg**, **frac**, **npdf** and the rest—apply elementwise. Note that **inv** is therefore the elementwise reciprocal; the matrix inverse is **minv**.

7.3 Linear algebra

tran, **'**	`A -- A^T`	transpose
minv	`A -- A^-1`	inverse of a square real or complex matrix; singular is an error
pinv	`A -- A^+`	Moore–Penrose pseudoinverse (real, square), by SVD
det	`A -- det(A)`	determinant
eig	`A -- V L`	eigenvectors (columns of V) and eigenvalues (diagonal of L), both complex
chol	`A -- L`	Cholesky factor of a symmetric positive definite matrix, $A = LL^T$
svd	`A -- U S V`	singular value decomposition, $A = USV^T$
kron	`A B -- A(x)B`	Kronecker product

```
[2 2 $ 4 7 2 6] det            \ 10
[2 2 $ 4 7 2 6] minv pm        \ |  0.6 -0.7 |
                               \ | -0.2  0.4 |
[2 2 $ 2 1 1 2] eig pm         \ L: eigenvalues 3 and 1 on the diagonal
[2 2 $ 4 2 2 3] chol pm        \ | 2 0 | / | 1 1.4142 |
```

Solving $Ax = b$ is `b A minv *`—or, keeping the operands in the natural order, `A b swap minv *`. The `std` word module adds `cond` (condition number), `norm2`, `normfro` and `XprimeX` (Appendix B).

7.4 Elements and shape

Indices are 0-based.

dim	`A -- A rows cols`	dimensions; the matrix stays
get_aij	`A i j -- A a_ij`	element (i, j); the matrix stays
set_aij	`A i j x -- A`	set element (i, j) in place; a complex x promotes a real matrix
reshape	`A rows cols -- B`	reshape in row-major order
split_mat	`A -- a11 a12 ...`	push every element as a scalar, row by row
diag	`A -- v`	the main diagonal as a row vector
to_diag	`v -- D`	diagonal matrix from a row or column vector
join_v	`A B -- [A; B]`	stack vertically (same number of columns)
join_h	`A B -- [A B]`	join side by side (same number of rows)

```
[2 2 $ 1 2 3 4] 0 1 get_aij        \ A 2
[2 2 $ 1 2 3 4] 1 0 9 set_aij pm   \ | 1 2 | / | 9 4 |
```

```
[2 3 $ 1 2 3 4 5 6] 3 2 reshape pm \ | 1 2 | / | 3 4 | / | 5 6 |
[1 2 $ 1 2] [1 2 $ 3 4] join_v pm  \ | 1 2 | / | 3 4 |
```

7.5 Reductions and statistics

The `c`-prefixed commands reduce each column and return a row vector; the `r`-prefixed ones reduce each row and return a column vector. They consume the matrix: **`csum`**/**`rsum`**, **`cmean`**/**`rmean`**, **`cvar`**/**`rvar`** (sample variance, $n-1$ in the denominator), **`cmin`**/**`rmin`**, **`cmax`**/**`rmax`**. **`cumsum_r`** and **`cumsum_c`** are cumulative sums along rows and down columns; `cumprod_r` and `cumprod_c` in the `std` module do the same for products.

```
[2 2 $ 1 2 3 4] cmean pm       \ | 2 3 |
[2 2 $ 1 2 3 4] rsum pm        \ | 3 | / | 7 |
[2 2 $ 1 2 3 4] cvar pm        \ | 2 2 |
```

7.6 Polynomials

A polynomial is a vector of coefficients, *highest degree first*, as in Matlab. **`pval`** evaluates it at a real or complex point; **`roots`** returns all roots as a complex row vector.

```
[1 3 $ 1 -3 2] 2 pval          \ 0        x^2 - 3x + 2 at x = 2
[1 3 $ 1 -3 2] roots pm        \ | (1,0) (2,0) |
[1 3 $ 1 0 1] roots pm         \ | (0,1) (0,-1) |   x^2 + 1
```

The `npv` and `irr` words of the `std` module are built on these two commands (Appendix B).

8 Strings

`scon`	`s1 s2 -- s1s2`	concatenate
`substr`	`s start end -- sub`	substring, 0-based, end exclusive
`slen`	`s -- n`	length in bytes
`s2u`, **`s2l`**	`s -- s`	upper and lower case
`srev`	`s -- s`	reverse
`int2str`	`x -- s`	integer part of x as a string
`eval`	`s --`	evaluate the string as a line of input

```
"abcdef" 1 3 substr        \ "bc"
"foo" "bar" scon s2u       \ "FOOBAR"
"2 3 +" eval               \ 5
"12:45" dec_hr             \ 12.75   (from the std module: substr + eval)
```

`eval` is the bridge from text to computation: the `std` word `dec_hr` converts a time string such as the one **`sunrise`** returns into decimal hours by cutting out the hour and minute fields with **`substr`** and evaluating them.

9 Registers

There are 64 storage registers, numbered 0–63, and each can hold any type—a matrix or a string as well as a number. Following the HP convention the value comes first and the register number last:

```
42 0 sto            \ store 42 in R0
0 rcl               \ 42   (a copy; the register keeps its value)
[2 2 $ 1 2 3 4] 5 sto
pr                  \ shows which registers are occupied
ffr                 \ 1    index of the first free register
```

`sto_add`, `sto_sub`, `sto_mul` and `sto_div` (in the `std` module) are the HP storage arithmetic: `5 0 sto_add` adds 5 to register 0. **`saveregs`** and **`loadregs`** write and read `registers.txt` in the configuration directory; **`clregs`** empties all registers.

Registers are the natural way to pass parameters to a word used in a numerical method (Section 11): the word reads the parameter with **`rcl`** and you change it with **`sto`** between calls.

10 Words: programming the calculator

A *word* is a named sequence of tokens. Defining one takes a single line that starts with a colon and ends with a semicolon:

```
: sq dup * ;
5 sq                \ 25
: hyp sq swap sq + sqrt ;
3 4 hyp             \ 5
```

Once defined, a word is used exactly like a built-in command; it can appear in other words, in scripts, in programs, and as a function for the numerical methods. **`listwords`** lists your words with their definitions, `"hyp" usage` shows one, `"hyp" delword` deletes one, and **`clrwords`** deletes them all.

Words are *compiled* when defined. Every name in the body must already exist (as a command or a word, from any module), so a typo is reported immediately rather than when the word is first run; the control structures described below are turned into branches at this point. A word may call itself, and nesting stops with an error at 256 levels. Redefining a word replaces it, and any word that calls it sees the new definition.

A name must not be a built-in command or control word and must not start with a digit. Names are case sensitive.

10.1 Conditionals

Inside a word, **`if`** consumes a flag from the stack and executes the following tokens when it is true (non-zero); **`else`** introduces the alternative; **`then`** closes the construct.

```
: abs2 dup 0 lt if chs then ;
-3 abs2             \ 3
: sign dup 0 lt if drop -1 else 0 gt if 1 else 0 then then ;
-7 sign 0 sign 2 sign     \ -1 0 1
```

10.2 Loops

`begin ... flag until`
repeat until the flag is true (the body runs at least once).

`begin ... flag while ... repeat`
test in the middle; leave the loop when the flag is false.

`limit start do ... loop`
a counted loop from `start` up to but not including `limit`; **`i`** is the current index and **`j`** the index of the enclosing loop. If `start` equals `limit` the body is skipped.

`limit start do ... n +loop`
the same with a step of n; a negative step counts down to `limit` inclusive.

Loops nest to 16 levels. Flags are reals; 0 is false.

```
: fac dup 1 gt if dup 1 - fac * then ;         \ recursive
10 fac                                        \ 3628800

: sum 0 swap 1 + 1 do i + loop ;              \ 1 + 2 + ... + n
100 sum                                       \ 5050

: step dup 2 mod if 3 * 1 + else 2 / then ;   \ one Collatz step
: collatz 0 swap begin dup 1 gt while swap 1 + swap step repeat drop ;
27 collatz                                    \ 111 steps

: fib 0 swap 1 swap 0 do tuck + loop drop ;   \ nth Fibonacci number
30 fib                                        \ 832040

: table 1 + 1 do i dup sq loop ;              \ i and i^2 for i = 1..n
3 table                                       \ 1 1 2 4 3 9
```

A definition occupies the whole line: nothing may follow the closing semicolon. Long bodies are best split into words, as `collatz` does with `step`.

10.3 Saving words

`savewords` writes all your words to `user_words.txt` in the configuration directory, one `name body` per line, and they are loaded automatically at the next start; **`loadwords`** reloads the file by hand. The file is plain text and you may edit it: forward references are fine (it is loaded in two passes), comments start with `\`, and a line that fails to compile is reported with its line number and skipped.

10.4 Words from modules

Words also arrive in *word modules*: files of definitions plugged in at startup, such as the shipped `std` module (Appendix B). They behave exactly like your own words—the only difference is where they are stored, so that **`clrwords`** does not remove them. **`listmacros`** lists them by module. If one of your words has the same name as a module's word, yours wins. Section 15 explains how to write a module.

11 Numerical methods

The numerical methods take a function as an execution token—`@name` for any word or command—and treat it as a map $x \mapsto f(x)$: the function is called with x on the stack and must leave exactly one real number. An error inside the function aborts the method, and the stack is restored.

integrate	`a b xt -- I`	$\int_a^b f$ by tanh–sinh quadrature with adaptive bisection
romberg	`a b xt -- I`	$\int_a^b f$ by Romberg extrapolation
fzero	`a b xt -- x`	root in a bracket $[a, b]$ where f changes sign (Brent)
solve	`g xt -- x`	root near one guess g (HP-15C style: expand a bracket, then Brent)
minimize	`a b xt -- x`	argument of the minimum of f on $[a, b]$ (Brent)
deriv	`x xt -- f'(x)`	derivative by Ridders' extrapolation
fsum, **fprod**	`lo hi xt -- S`	$\sum f(i)$ or $\prod f(i)$ over the integers $lo..hi$
exec	`xt --`	run the token
tol	`x --`	set the tolerance, relative to $\max(1, \lvert\text{result}\rvert)$; default 10^{-10}
nevals	`-- n`	function evaluations used by the last call
interr	`-- e`	error estimate of the last **integrate**

```
12 setprec
0 2 @exp integrate              \ 6.389056098931   (e^2 - 1)
: rsqrt sqrt inv ;
0 1 @rsqrt integrate            \ 2.000000000000   endpoint singularity, exact
0 pi @sin integrate             \ 2.000000000000
0 1 @exp romberg                \ 1.718281828459
2 4 @sin fzero                  \ 3.141592653605
3 @sin solve                    \ 3.141592653590
: par 3 - dup * 1 + ;
0 5 @par minimize               \ 3.000000000000
2 @exp deriv                    \ 7.389056098931
: invsq dup * inv ;
1 1000 @invsq fsum              \ 1.643934566682   (pi^2/6 - 1/1000 + ...)
3 @par exec                     \ 1
```

Some practical notes.

- **integrate** uses the tanh–sinh rule, whose nodes cluster at the endpoints; an integrable singularity *at* an endpoint converges to full precision, and the method is exponentially convergent for analytic integrands. Interior kinks or peaks are handled by bisecting the interval where the rule does not converge. If the error estimate stays above the tolerance a warning is printed and the result is returned anyway; **interr** tells you how good it is.
- **fzero** needs a sign change; **solve** needs only a guess and will grow a bracket around it, so it is the one to reach for interactively. Both are exact to the tolerance for simple roots.
- A function may take parameters from registers: `: f 0 rcl * sin ;` integrates $\sin(kx)$ with k in register 0, and `2 0 sto` changes k.
- The function may be a built-in (`@sin`, `@exp`) or a word from a module as well as your own word. It may also be a long-running word; the limit is two million evaluations per call, after which the method gives up with an error suggesting a looser tolerance.

- Loosen **tol** for expensive functions: `1e-6 tol`. The value is kept in `config.txt`.

The methods were cross-checked against SciPy on 62 cases with a worst relative error of 3.5×10^{-11}.

12 Dates, time and the sun

Dates are strings in the form `d.m.yyyy`; leading zeros are optional on input and present on output (`"1.2.2025"` and `"01.02.2025"` are the same date). Times are strings `HH:MM`.

today	`-- date`	today's date
now	`-- time`	current local time
dateplus	`date n -- date`	n days later (n may be negative)
ddays	`d1 d2 -- n`	days from `d1` to `d2`
dow	`date -- name`	weekday name
edmy	`date -- y m d`	split into year, month, day
num2date	`d m y -- date`	build a date
days2eoy	`date -- n`	days from the date to the end of its year
tz_offset	`date zone -- h`	UTC offset in hours of an IANA zone on that date
sunrise, **sunset**	`date lat lon zone -- time`	local sunrise and sunset
dawn, **dusk**	`date lat lon zone -- time`	civil dawn and dusk (sun $6°$ below the horizon)
workdays …		workdays and holidays, Section 12.1

```
"1.1.2025" 30 dateplus         \ "31.01.2025"
"1.1.2025" "31.12.2025" ddays  \ 364
"1.1.2025" dow                 \ "Wednesday"
"1.3.2024" days2eoy            \ 305
"1.12.2026" "America/New_York" tz_offset        \ -5
"21.6.2026" 46.24 14.36 "Europe/Ljubljana" sunset   \ "20:58"
```

The `zone` argument of the solar functions is either an IANA zone name such as `"Europe/Ljubljana"` or a UTC offset in hours such as `-4`. With a zone name the offset is resolved *for the date on the stack*, so daylight-saving time is never your problem; names are validated against the system's `zoneinfo` database. The location words of the `std` module push latitude, longitude and zone in one go:

```
today 100 dateplus durham dawn      \ civil dawn in Durham, NC, 100 days from now
today kranj sunset dec_hr           \ sunset in Kranj as a decimal hour
```

Add your own with one line in `user_words.txt` or in a word module of your own: `home 51.5 -0.12 "Europe/London"`.

12.1 Workdays and holidays

A workday is a Monday to Friday that is not in the *holiday calendar*, a list of dates the dates module keeps for the session. These words take dates as strings `"d.m.yyyy"` or as numbers `YYYYMMDD`, like the finance words, and follow Excel's conventions so that the results can be checked against a spreadsheet.

workdays	`d1 d2 -- n`	workdays from `d1` to `d2`, both ends inclusive (`NETWORKDAYS`); negative when `d2` precedes `d1`
workdayplus	`date n -- date2`	the date n workdays after `date`, the start day itself not counted (`WORKDAY`); n may be negative
isworkday	`date -- flag`	1 for a workday
holidays	`name -- H`	a calendar as a vector of `YYYYMMDD`: the file `holidays/name.txt` in the configuration directory, or the built-in rules for the years around today
yholidays	`year name -- H`	the built-in rules for one year
sethol	`H --`	install a vector of dates as the calendar for this session
usehol	`name --`	install a calendar by name and remember it in `config.txt`
clrhol	`--`	no holidays (weekends only), and forget the name
hol	`-- H`	the installed calendar

Until a calendar is installed only weekends are skipped:

```
"1.1.2026" "31.12.2026" workdays     \ 261
"5.1.2026" "1.1.2026" workdays       \ -3
"us" usehol                          \ the federal holidays, kept in config.txt
"1.1.2026" "31.12.2026" workdays     \ 250
"24.12.2026" 10 workdayplus          \ "11.01.2027"   (25.12 and 1.1 skipped)
20261126 isworkday                   \ 0              Thanksgiving
2027 "si" yholidays pm               \ Slovenia's work-free days of 2027
```

Two calendars are built in. `"us"` is the eleven federal holidays with the federal observance rule (a holiday on a Saturday is taken on the Friday before, on a Sunday on the Monday after, so New Year's Day 2028 appears as 31 December 2027). `"si"` is Slovenia's work-free days—fixed dates plus Easter Monday—which are not shifted. Without a year, **holidays** and **usehol** generate the rules for last year through two years ahead, which covers what you compute day to day; **yholidays** gives any single year from 1583.

Any other calendar is a file, `~/.config/mm41/holidays/`*name*`.txt`, with one date per line, `d.m.yyyy` or `YYYYMMDD`, anything after the date a comment, and `#` lines and blank lines ignored. `make install` puts an annotated example there, `holidays/example.txt`, which is an office calendar for one year (the federal holidays plus the day after Thanksgiving): copy it to `imf.txt`, edit, and `"imf" usehol`. A file calendar knows only the years you have typed into it; a date outside them is treated as having no holidays, so extend the file each year. A name containing a slash or ending in `.txt` is taken as a path, which is how a calendar can live anywhere.

The calendar named in `config.txt` (`holidays = us`) is loaded on first use. If it cannot be loaded—a misspelt name, a deleted file—every workday word reports the problem until you fix it, **clrhol**, or **usehol** another: a silently wrong count would be worse than an error. A holiday that falls on a weekend changes nothing, duplicates are dropped, and the list is sorted, so **hol** is also a tidy way to look at a calendar.

13 Finance

The `finance` module is MM-B1, the decimal business calculator, rebuilt as a module: the time value of money, amortisation, cash flows, interest-rate conversions, depreciation, bonds and day counts. Three conventions hold throughout.

- **Everything is decimal.** Every finance word computes in 34-digit decimals and leaves a decimal, whatever it was given; a real argument is converted on the way in. You can therefore use the module in either number mode, and `2 setprec` shows money to the cent, correctly rounded.
- **Rates are in percent, per period.** `1 i%` is one percent per period; a 6 % annual rate paid monthly is `0.5 i%`. The dated words (**`xnpv`**, **`xirr`**, **`xprice`**, ...) take annual rates.
- **Cash received is positive, cash paid is negative.** A loan is a positive `pv` with a negative `pmt`; a savings plan is a negative `pmt` with a positive `fv`.

13.1 Time value of money

Like the HP-12C, the module keeps five registers—n (periods), $i\%$ (rate per period), pv, pmt and fv—tied together by

$$pv\,(1+r)^n + pmt\,(1+r\,\tau)\,\frac{(1+r)^n-1}{r} + fv = 0, \qquad r = \frac{i\%}{100},$$

where τ is 0 when payments fall at the end of each period (**`end`**, the default) and 1 when they fall at the beginning (**`beg`**). Store a value with the register's name, solve for the unknown with a question mark:

`n i% pv pmt fv`	`x --`	store into the register
`?n ?i% ?pv ?pmt ?fv`	`-- x`	solve for that register from the other four; the result is stored and pushed
`beg`, **`end`**	`--`	payments at the beginning or the end of the period
`clrtvm`	`--`	clear the registers and select **`end`**
`tvm`	`--`	show the registers

```
2 setprec
clrtvm 360 n 0.5 i% 300000 pv 0 fv ?pmt      \ -1798.65   30-year mortgage at 6%
clrtvm 120 n 0.4 i% 0 pv -200 pmt ?fv        \ 30726.39   200 a month for 10 years at 4.8%
clrtvm 6 12 / i% 250000 pv -1500 pmt 0 fv ?n \ 359.25     months to repay
clrtvm 48 n 20000 pv -450 pmt 0 fv ?i% 12 *  \ 3.82       annual rate of a car loan
```

`?n`, **`?pv`**, **`?pmt`** and **`?fv`** are closed forms; **`?i%`** brackets the rate on a grid from -99 % to 1000 % per period and bisects in decimal to 34 digits, so it is exact to the last displayed digit and reports an error if no rate satisfies the equation. The registers are not saved between sessions.

13.2 Amortisation

`amort` prints the schedule of the loan in the registers, period by period: payment, interest, principal and remaining balance, with totals, all to the current display precision. **`amortm`** pushes the same schedule as an $n \times 4$ real matrix, for **`csum`**, **`getrow`** or saving to a file. Both use the magnitudes of `pv` and `pmt` and the **`end`** convention.

```
clrtvm 3 n 1 i% 1000 pv 0 fv ?pmt drop amort
  per        payment       interest      principal        balance
  --- -------------- -------------- -------------- --------------
    1         340.02          10.00         330.02         669.98
    2         340.02           6.70         333.32         336.66
    3         340.02           3.37         336.66           0.00
  --- -------------- -------------- -------------- --------------
  tot                        20.07        1000.00
```

13.3 Cash flows: NPV and IRR

Cash flows are given as a vector in time order, the first element at $t = 0$ (the outlay) and one element per period. When a flow repeats, an $N \times 2$ matrix gives the amounts in the first column and the repeat counts in the second.

npv	`cf rate -- x`	net present value at `rate` percent per period
irr	`cf -- x`	internal rate of return, percent per period

```
[5 1 $ -5000 1500 1500 1500 1500] 8 npv     \ -31.81    not worth it at 8%
[5 1 $ -5000 1500 1500 1500 1500] irr       \ 7.71
[2 2 $ -5000 1 1500 4] irr                  \ 7.71      the same flows, with a count
```

MM-15's `std` word module defined `npv` and `irr` as macros over **`pval`** and **`roots`**, with the cash flows written *latest first* and the rate as a fraction. Those macros are gone; the finance words take the flows in time order and the rate in percent, like every other business calculator.

13.4 Interest-rate conversions and depreciation

nom>eff	`nom m -- eff`	nominal annual rate compounded m times a year to the effective annual rate
eff>nom	`eff m -- nom`	the inverse
sl	`cost salvage life -- d`	straight-line depreciation per period
soyd	`cost salvage life year -- d`	sum-of-the-years'-digits depreciation in `year`
db	`cost salvage life year factor -- d`	declining-balance depreciation in `year`; `factor` 2 is double-declining; never below salvage

```
6 12 nom>eff           \ 6.17   6% monthly is 6.17% effective
6 365 nom>eff          \ 6.18
12.6825 12 eff>nom     \ 12.00
10000 1000 5 sl        \ 1800
10000 1000 5 1 soyd    \ 3000   first year
10000 1000 5 1 2 db    \ 4000   first year, double declining
10000 1000 5 5 2 db    \ 296    fifth year: capped by the salvage value
```

13.5 Bonds

Two bond words work in periods, for a bond settled on a coupon date: price per 100 of face value from the coupon and yield per period, the number of periods and the redemption value, and the yield from the price.

```
3 3.5 20 100 bprice     \ 92.89   3% coupon, 3.5% yield, 20 periods
92.89 3 20 100 byield   \ 3.50
```

The dated words follow the SIA conventions used by Excel's `PRICE`, `YIELD` and `ACCRINT`: settlement and maturity dates, the annual coupon rate and yield in percent, the redemption value, the number of coupons per year (1, 2, 3, 4, 6 or 12) and a day-count *basis*:

0	30/360 (US, NASD)
1	actual/actual (ISDA)
2	actual/360
3	actual/365
4	30E/360 (European)

xprice	`settle maturity cpn yld redeem freq basis -- price`	clean price per 100
xyield	`settle maturity cpn price redeem freq basis -- yld`	annual yield from the clean price
xaccr	`settle maturity cpn freq basis -- accrued`	accrued interest per 100 at settlement
yearfrac	`d1 d2 basis -- f`	year fraction between two dates on a basis

```
4 setprec
20080215 20171115 5.75 6.5 100 2 0 xprice       \ 94.6344   (Excel's PRICE example)
20080215 20161115 5.75 95.04287 100 2 0 xyield  \ 6.5000
20080215 20171115 5.75 2 0 xaccr                \ 1.4375
"1.1.2007" "1.1.2008" 3 yearfrac                \ 1.0000    actual/365
20070101 20080101 2 yearfrac                    \ 1.0139    actual/360
```

13.6 Dates in the finance module

A date argument to any finance word is either a date string of the dates module, `"d.m.yyyy"`, or the integer `YYYYMMDD`, which is the form that fits in a matrix. **`date>ymd`** and **`ymd>date`** convert between the two, so the dates module's **`today`**, **`dateplus`** and **`ddays`** combine with the finance words:

```
"15.3.2026" date>ymd       \ 20260315
20260315 ymd>date          \ "15.03.2026"
today 90 dateplus date>ymd \ the numeric date 90 days from now
```

13.7 Dated cash flows: XNPV and XIRR

Irregularly spaced flows are an $N \times 2$ matrix of amount and numeric date, in date order. **`xnpv`** discounts each flow by actual/365 from the first date at an *annual* rate in percent, exactly as Excel's `XNPV`; **`xirr`** finds the annual rate that makes the net present value zero.

```
2 setprec
[4 2 $ -5000 20250110  2000 20250601  2000 20251201  2000 20260601] 8 xnpv   \ 605.77
[4 2 $ -5000 20250110  2000 20250601  2000 20251201  2000 20260601] xirr     \ 23.25
```

A matrix literal is binary, so the amounts are converted on the way in like every other real argument; dates that you hold as strings go through **`date>ymd`** first. The matrix can also be built up with **`join_v`** from 1×2 rows, or read from a file with `[4, 2, "flows.txt"]`.

14 Programs

Besides words, MM-41 runs HP-41 style *programs*: text files with one instruction per line, labels, jumps and subroutine calls. They are useful for longer procedures with explicit control flow, and they are how the calculator was programmed before words had loops.

`LBL name`
: a label.

`GOTO name`, `GOSUB name`, `RTN`
: jump, call, and return.

`END`
: stop (the program also stops at the end of the file).

tests
: a line consisting of a test such as `top_gt?` skips the next line when the test is false. The tests do not consume anything. With y below x on the stack: `top_eq?`, `top_neq?`, `top_lt?`, `top_leq?`, `top_gt?`, `top_gte?` compare y with x; `top_eq0?`, `top_neq0?`, `top_lt0?`, `top_lte0?`, `top_gt0?`, `top_gte0?` compare x with zero.

anything else
: is evaluated as a line of ordinary input, so a line may hold several tokens.

Blank lines and lines starting with `\` or `#` are ignored.

A factorial, `fact.prg`:

```
\ n -- n!
1
LBL loop
over 1
top_gt?
GOTO body
GOTO done
LBL body
drop *
swap 1 - swap
GOTO loop
LBL done
drop drop nip
END
```

```
5 "fact.prg" run        \ 120
```

run loads and runs a program file; with `verbose_mode = 1` in the configuration it also prints the listing. Two related commands: **batch** runs a file of ordinary lines (no labels or tests), and **eval** evaluates a string.

15 Modules

The HP-41 took plug-in ROMs; MM-41 takes modules. A module is a named set of commands with a version and a one-line summary. The core of the calculator—numbers, the stack, words and control flow, errors, batch mode—is itself the module `core`, and everything a user would call a feature is a module alongside it. **modules** shows what is plugged in:

```
MM-41>> modules
module       version  words  summary
core         1.2         85  arithmetic, functions, logic, the stack, words and the session
decimal      1.0          7  decimal numbers for money: number mode, conversions, rounding
finance      1.0         33  time value of money, cash flows, depreciation, bonds, day counts
complex      1.0          7  complex numbers: parts, argument, conjugate, construction
special      1.0          7  gamma, beta and the standard normal distribution
matrices     1.0         38  linear algebra, matrix construction and reduction, polynomials
numerics     1.0         12  integration, roots, minima, derivatives and sums over any word
strings      1.0          7  string construction and manipulation
registers    1.0          7  64 storage registers of any type
dates        1.1         22  calendar arithmetic, workdays and holidays, time zones, sunrise and sunset
programs     1.0          1  HP-41 style programs with labels, jumps and subroutines
std          1.0         28  standard word module: powers, norms, storage arithmetic, locations
```

Every command belongs to exactly one module; `"name" usage` says which. Two modules may not define the same name—a clash is reported when the second one loads, with the file and line for a word module—so a command means the same thing whatever is plugged in. There are three kinds of module, and from the stack they are indistinguishable.

15.1 Built-in modules

These are compiled into the calculator from the `modules/` directory of the source tree, one subdirectory each. The `MODULES` variable in the Makefile chooses which; the default is all of them, and

```
make MODULES="matrices strings"
```

builds a smaller calculator with only those two (and the core). The command reference in Appendix A is generated per module, so it always describes the binary it was built with.

15.2 Word modules

A word module is a text file of definitions, and it is the way to add commands without a compiler. The file holds one definition per line in either form, with an optional header line naming the module:

```
\ module geo 0.1 circles and spheres
: circ 2 * pi * ;             \ circumference of a circle of radius x
sphere dup * 4 * pi *         \ surface of a sphere (name body form)
```

Save it as `geo.mm` in `~/.config/mm41/modules/` and it is loaded at every start, like a ROM left in its slot; the `std` module ships there. To load one by hand:

```
"geo" use                 \ modules/geo.mm in the configuration directory
"/tmp/draft.mm" use       \ or any path
"geo" unuse               \ remove its words
listmacros                \ the words of every loaded word module, by module
```

Loading a module that is already loaded replaces it, which makes editing one a quick cycle of save and **`use`**. Definitions are compiled on loading with the same rules as words typed at the prompt: unknown names, control-flow mistakes and clashes are reported with the line number, the offending word is skipped, and the rest of the module loads. Forward references within a module are fine. Modules are loaded in name order, so a module that uses another's words should sort after it (or simply not depend on it).

Word modules cannot redefine a built-in command. They can be shadowed by your own words, which is deliberate: a word you type always wins.

15.3 Compiled modules

A compiled module is a shared object exporting one function, `mm41_module()`, which returns the same `Module` structure the built-in modules use (`include/module.h`):

```
#include "module.h"
#include "errors.h"
#include "stack.h"

static int op_cube(Stack *stack) {           /* x -- x^3 */
  double x;
  if (pop_real_arg(stack, "cube", &x) != 0) return -1;
  push_real(stack, x * x * x);
  return 0;
}

static const Op table[] = {
  /* name, min depth, function, category, stack picture, description, example */
  { "cube", 1, op_cube, "Example", "x -- x^3", "cube of a real number", "3 cube" },
};

static const Module mod = {
  "hello", "0.1", "example module",
  table, (int)(sizeof table / sizeof table[0]),
  NULL, NULL,           /* init, shutdown */
  MM41_MODULE_API
};

const Module *mm41_module(void) { return &mod; }
```

A command takes the stack and returns 0, or reports a problem with `mm_error()` and returns nonzero; the core checks the minimum depth before calling it and rolls the stack back on error, exactly as for a built-in. The helpers in `ops.h` (`pop_real_arg`, `pop_two_reals`, `pop_string_arg`) and the whole stack API are available. The complete example, with init and shutdown hooks and three commands, is `examples/modules/hello/`.

```
make shared-module DIR=examples/modules/hello     # -> hello.so
```

```
"examples/modules/hello/hello.so" load
3 cube                      \ 27
"cube" usage                \ [hello: Example]
```

A `.so` placed in `~/.config/mm41/modules/` is loaded at startup. A compiled module stays loaded for the session (its code must remain mapped while its commands are in the dispatch table), and loading one renumbers execution tokens, so do not keep an `@name` token on the stack across a **`load`**. The `api` field guards against a module built for a different version of the interface.

15.4 Which kind to write

A word module for anything that can be said in words—formulas, conventions, locations, a house style of abbreviations; it needs no compiler and travels as one text file. A compiled module when you need a new algorithm, a library, or speed: an FFT, a database connector, a different number type. The `std` module is the first kind; the matrix module is the second, and the only difference between it and a module you write is that it happens to be compiled in.

16 Batch mode and scripting

When MM-41 is given expressions or files on the command line, or when its standard input is not a terminal, it runs in *batch mode*: no banner, no per-line echo, the final stack is printed, and the exit status is 1 if any error occurred.

```
mm41 -e "2 10 ^"                        # one line
mm41 -e "[2 2 $ 4 7 2 6] det" -e "sqrt"  # several, in order
mm41 -f script.rpn                      # a file; '-' means stdin
echo "1 3 /" | mm41 -n                  # a pipeline
mm41 -n -e ': f dup * 2 - ;' -e '0 3 @f fzero'
```

`-n` starts with a clean slate: no saved words, modules or configuration, and nothing is written at exit. Use it in scripts so that their behaviour does not depend on the state of your configuration directory. Scripts may define words, and `\` or `#` comments are allowed. The project's own tests are batch runs compared against golden output files (`tests/golden/`).

`-e EXPR`	evaluate one line; may be repeated
`-f FILE`	run the lines of a file; `-` is standard input; may be repeated and mixed with `-e`
`-n`	do not load the saved words, the modules in the config directory, or the configuration
`--modules`	list the built-in modules and exit
`--commands`	print the command reference as Markdown
`--commands-tex`	the same as LaTeX tables (used to build this manual)
`--version`, `-h`	version; usage

Exit status: 0 on success, 1 if any error was reported in batch mode, 2 for a usage error.

17 Configuration and files

Everything lives in `$XDG_CONFIG_HOME/mm41/`, which defaults to `~/.config/mm41/`:

`config.txt`	`print_precision`, `fixed_point`, `number_mode`, `holidays`, `verbose_mode`, `tol`; written at exit
`holidays/*.txt`	holiday calendars (Section 12.1); the shipped `example.txt` is replaced by `make install`, your own files are kept
`modules/*.mm`	word modules, loaded at startup in name order; the shipped ones are replaced by `make install`
`modules/*.so`	compiled modules, loaded at startup
`user_words.txt`	words saved by **`savewords`**; loaded at startup
`registers.txt`	**`saveregs`** / **`loadregs`**
`stack_backup.dat`	**`savestack`** / **`loadstack`**
`.rpn_history`	readline history (last 1000 lines)

A missing configuration directory is created on first use. `verbose_mode = 1` makes the calculator chattier: it lists the module words at startup, echoes word definitions in batch mode, and prints program listings.

The configuration directory is per user; the `data/` directory in the source tree holds the defaults that `make install` copies there if nothing exists yet.

18 Errors

Every top-level token is atomic. When a command fails, the stack is restored to its state before that token, a message beginning with `Error:` is printed on standard error, and the rest of the line is skipped:

```
MM_RPN>> 1 2 [2 2 $ 1 2 2 4] minv +
Error: Matrix is singular, cannot invert
[0] ℝ : 1.000000
[1] ℝ : 2.000000
[2] 𝕄ℝ: 2 x 2 matrix
```

The **+** after the failing **minv** did not run. A user word counts as one token: if it fails half-way, nothing it did remains. **undo** goes one step further back and restores the stack as it was before the previous line.

Common messages:

needs N item(s) on the stack a command was given too few arguments.

unsupported operand types e.g. adding a string to a number, or multiplying matrices of incompatible shapes.

Division by zero from **/**, **inv** or **./**.

must leave exactly one real result a function passed to a numerical method pushed nothing, several items, or a non-real.

runaway recursion a word called itself more than 256 levels deep.

In batch mode any error makes the exit status 1, so a script can be checked with && like any other command.

A Command reference

Generated from the dispatch table by `mm41 --commands-tex`; the same information is available at run time from **`listfcns`** and `"name" usage`. Stack pictures read deepest-first.

Module `core` 1.2 — arithmetic, functions, logic, the stack, words and the session

Arithmetic

Command	Stack	Description	Example
`+`	`y x -- y+x`	add scalars or matrices; a scalar added to a matrix is added elementwise	`3 4 +`
`-`	`y x -- y-x`	subtract scalars or matrices (a scalar elementwise)	`10 4 -`
`*`	`y x -- y*x`	multiply scalars, or matrix product for matrices	`[2 2 $ 1 2 3 4] [2 1 $ 1 1] *`
`/`	`y x -- y/x`	divide scalars, or a matrix by a scalar; division by zero is an error	`1 3 /`
`^`	`y x -- y^x`	power; a square matrix to an integer power is a matrix power; a negative base with a fractional exponent promotes to complex	`2 10 ^`
`pow`	`y x -- y^x`	power (alias of ^)	`2 10 pow`
`.*`	`A B -- A.*B`	elementwise matrix product	`[2 2 $ 1 2 3 4] [2 2 $ 1 1 2 2] .*`
`./`	`A B -- A./B`	elementwise division (matrix or scalar operands)	`[2 2 $ 1 2 3 4] [2 2 $ 1 2 1 2] ./`
`.^`	`A B -- A.^B`	elementwise power (matrix or scalar operands)	`[2 2 $ 1 2 3 4] 2 .^`
`chs`	`x -- -x`	change sign (elementwise for matrices)	`5 chs`
`inv`	`x -- 1/x`	reciprocal (elementwise for matrices; see minv for the matrix inverse)	`4 inv`
`abs`	`z -- |z|`	absolute value; magnitude for complex; elementwise for matrices	`(3,4) abs`
`sqrt`	`x -- sqrt(x)`	square root; a negative real promotes to complex	`-4 sqrt`
`%`	`y x -- y y*x/100`	percent of y (HP %; keeps y)	`200 25 %`
`%t`	`y x -- y 100*x/y`	x as a percent of the total y (HP %T; keeps y)	`200 50 %t`
`%chg`	`y x -- y 100*(x-y)/y`	percent change from y to x (HP delta%; keeps y)	`80 100 %chg`
`mod`	`y x -- y mod x`	floored modulo (result has the sign of x)	`-7 3 mod`
`fact`	`x -- x!`	factorial; Gamma(x+1) for non-integers	`5 fact`
`ncr`	`n k -- C(n,k)`	combinations	`52 5 ncr`
`npr`	`n k -- P(n,k)`	permutations	`10 3 npr`
`frac`	`x -- frac(x)`	fractional part, x - intg(x), with the sign of x	`3.7 frac`

Command	Stack	Description	Example
intg	x -- int(x)	integer part, truncated toward zero as on HP calculators (-3.7 gives -3)	3.7 intg

Exp/Log

Command	Stack	Description	Example
exp	x -- e^x	exponential (elementwise for matrices)	1 exp
ln	x -- ln(x)	natural logarithm; a negative real promotes to complex	10 ln
log	x -- log10(x)	base-10 logarithm; a negative real promotes to complex	100 log

Trigonometry

Command	Stack	Description	Example
sin	z -- sin(z)	sine of a real or complex argument (radians); elementwise for matrices	0.5 sin
cos	z -- cos(z)	cosine of real or complex argument	1 cos
tan	z -- tan(z)	tangent of real or complex argument	0.3 tan
asin	z -- asin(z)	inverse sine (principal value)	0.5 asin
acos	z -- acos(z)	inverse cosine (principal value)	0.5 acos
atan	z -- atan(z)	inverse tangent (principal value)	1 atan
sinh	z -- sinh(z)	hyperbolic sine	1 sinh
cosh	z -- cosh(z)	hyperbolic cosine	1 cosh
tanh	z -- tanh(z)	hyperbolic tangent	1 tanh
asinh	z -- asinh(z)	inverse hyperbolic sine	1 asinh
acosh	z -- acosh(z)	inverse hyperbolic cosine	2 acosh
atanh	z -- atanh(z)	inverse hyperbolic tangent	0.5 atanh

Constants

Command	Stack	Description	Example
pi	-- π	push π	pi
e	-- e	push Euler's number	e
gravity	-- g	standard gravity, 9.80665 m/s^2	gravity
inf	-- +∞	positive infinity	inf
nan	-- NaN	not-a-number	nan

Logic

Command	Stack	Description	Example
eq	a b -- flag	1 if a equals b, else 0; numbers, strings, or elementwise for matrices	2 2 eq
neq	a b -- flag	1 if a differs from b; numbers, strings, or elementwise for matrices	2 3 neq
lt	a b -- flag	1 if a < b (complex numbers compare by magnitude); elementwise for matrices	2 3 lt
leq	a b -- flag	1 if a <= b; elementwise for matrices	2 3 leq
gt	a b -- flag	1 if a > b; elementwise for matrices	3 2 gt
geq	a b -- flag	1 if a >= b; elementwise for matrices	3 2 geq
and	a b -- flag	logical and (0 is false, anything else true)	1 0 and
or	a b -- flag	logical or	1 0 or
not	a -- flag	logical not	0 not

Stack

Command	Stack	Description	Example
dup	x -- x x	duplicate top of stack	5 dup
drop	x --	drop the top stack element	1 2 drop
swap	a b -- b a	swap top two elements	1 2 swap
over	a b -- a b a	copy second item to top	1 2 over
nip	a b -- b	drop second-from-top element (keep top)	1 2 nip
tuck	a b -- b a b	duplicate second item and tuck below top	1 2 tuck
roll	... n -- ...	move the item at depth n to the top (1 roll = swap, 2 roll = rot)	1 2 3 2 roll
pick	... n -- ... v	copy the item at depth n to the top (0 pick = dup)	1 2 3 2 pick
depth	-- n	number of items on the stack	depth
clst	... --	clear the stack	clst
savestack	--	save the whole stack to stack_backup.dat in the config directory	savestack
loadstack	-- ...	push the stack saved by savestack	loadstack

Display

Command	Stack	Description	Example
print	x -- x	print the top of the stack (a matrix in full)	42 print
pm	A -- A	print the matrix on top of the stack in full	[2 2 $ 1 2 3 4] pm
ps	--	print the whole stack	ps
setprec	n --	digits after the decimal point (fixed) or significant digits (scientific), 0-20	10 setprec

Command	Stack	Description	Example
`sfs`	`--`	toggle between fixed-point and scientific display	`sfs`

Words

Command	Stack	Description	Example
`listwords`	`--`	list the user-defined words	`listwords`
`listmacros`	`--`	list the words of the loaded word modules	`listmacros`
`loadwords`	`--`	load the words saved in user_words.txt (also done at startup)	`loadwords`
`savewords`	`--`	save all user-defined words to user_words.txt in the config directory	`savewords`
`delword`	`"name" --`	delete a user-defined word	`"sq" delword`
`clrwords`	`--`	delete all user-defined words	`clrwords`
`use`	`"name" --`	load a word module: modules/name.mm in the config directory, or a path	`"finance" use`
`unuse`	`"name" --`	unload a word module and its words	`"finance" unuse`

Session

Command	Stack	Description	Example
`load`	`"file.so" --`	load a compiled module (a shared object exporting mm41_module); stays loaded	`"hello.so" load`
`eval`	`s --`	evaluate the string s as a line of input	`"1 2 +" eval`
`batch`	`s --`	run the lines of file s as input	`"calc.txt" batch`
`help`	`--`	print the quick-start help screen	`help`
`usage`	`"name" --`	show the stack picture, description and example of a command, or a word's definition	`"sin" usage`
`listfcns`	`--`	list all commands by category	`listfcns`
`clrhist`	`--`	clear the readline history	`clrhist`
`fuck`	`--`	an Easter egg	`fuck`
`modules`	`--`	list the loaded modules with their versions	`modules`

Module `decimal` 1.0 — decimal numbers for money: number mode, conversions, rounding

Decimal

Command	Stack	Description	Example
`decimal`	`--`	number mode: literals are decimals (34 digits, half-even) until 'binary'	`decimal 0.1 0.2 +`
`binary`	`--`	number mode: literals are binary doubles (the default)	`binary`
`>dec`	`x -- d`	convert a real to a decimal by its shortest round-trip form (0.1 stays 0.1)	`0.1 >dec`
`>bin`	`d -- x`	convert a decimal to a binary real (needed before matrices and the numerical methods)	`1d 3d / >bin`
`round`	`x n -- d`	round to n decimal places, half-even; the result is a decimal	`2.345d 2 round`
`decmode`	`-- flag`	1 if the number mode is decimal	`decmode`
`isdec`	`x -- flag`	1 if x is a decimal	`1d isdec`

Module `finance` 1.0 — time value of money, cash flows, depreciation, bonds, day counts

Finance

Command	Stack	Description	Example
`n`	`x --`	store the number of periods in the TVM register n	`12 n`
`i%`	`x --`	store the interest rate per period, in percent, in i%	`1 i%`
`pv`	`x --`	store the present value (cash received +, paid -)	`10000 pv`
`pmt`	`x --`	store the periodic payment	`-888.49 pmt`
`fv`	`x --`	store the future value	`0 fv`
`beg`	`--`	payments fall at the beginning of each period (annuity due)	`beg`
`end`	`--`	payments fall at the end of each period (the default)	`end`
`clrtvm`	`--`	clear the five TVM registers and select 'end'	`clrtvm`
`tvm`	`--`	show the TVM registers	`tvm`
`?n`	`-- n`	solve for the number of periods; the result is stored and pushed (a decimal)	`clrtvm 1 i% 10000 pv -888.49 pmt 0 fv ?n`
`?i%`	`-- i`	solve for the rate per period in percent (bracketing scan, then bisection)	`clrtvm 12 n 10000 pv -888.49 pmt 0 fv ?i%`
`?pv`	`-- pv`	solve for the present value	`clrtvm 12 n 1 i% -888.49 pmt 0 fv ?pv`
`?pmt`	`-- pmt`	solve for the periodic payment	`clrtvm 12 n 1 i% 10000 pv 0 fv ?pmt`
`?fv`	`-- fv`	solve for the future value	`clrtvm 12 n 1 i% 0 pv -100 pmt ?fv`
`amort`	`--`	print the amortisation schedule of the TVM registers (n periods of pmt on pv at i%)	`clrtvm 12 n 1 i% 10000 pv 0 fv ?pmt drop amort`

Command	Stack	Description	Example
`amortm`	`-- M`	the amortisation schedule as an n×4 real matrix: payment, interest, principal, balance	`clrtvm 12 n 1 i% 10000 pv 0 fv ?pmt drop amortm`
`npv`	`cf rate -- x`	net present value of cash flows (a vector in time order, first at t=0, or N×2 amount/count) at rate percent per period	`[6 1 $ -1000 300 300 300 300 300] 10 npv`
`irr`	`cf -- x`	internal rate of return in percent per period of a cash-flow vector	`[2 2 $ -1000 1 300 5] irr`
`nom>eff`	`nom m -- eff`	nominal annual rate compounded m times a year to the effective annual rate (percent)	`12 12 nom>eff`
`eff>nom`	`eff m -- nom`	effective annual rate to the nominal rate compounded m times a year	`12.6825 12 eff>nom`
`sl`	`cost salvage life -- d`	straight-line depreciation per period	`10000 1000 5 sl`
`soyd`	`cost salvage life year -- d`	sum-of-the-years'-digits depreciation in the given year	`10000 1000 5 1 soyd`
`db`	`cost salvage life year factor -- d`	declining-balance depreciation in the given year (factor 2 = double declining), never below salvage	`10000 1000 5 1 2 db`
`bprice`	`cpn yld n redeem -- price`	bond price per 100 face on a coupon date: coupon and yield in percent per period, n periods, redemption value	`3 3.5 20 100 bprice`
`byield`	`price cpn n redeem -- yld`	bond yield in percent per period from the price on a coupon date	`92.89 3 20 100 byield`
`xnpv`	`M rate -- x`	net present value of dated cash flows (N×2 matrix: amount, date YYYYMMDD) at an annual rate in percent, actual/365 from the first date	`[5 2 $ -10000 20080101 2750 20080301 4250 20081030 3250 20090215 2750 20090401] 9 xnpv`
`xirr`	`M -- x`	annual internal rate of return in percent of dated cash flows	`[3 2 $ -1000 20250101 600 20250701 600 20260101] xirr`
`xprice`	`settle maturity cpn yld redeem freq basis -- price`	clean price per 100 face of a dated bond: annual coupon and yield in percent, coupons per year, day-count basis 0..4	`20080215 20171115 5.75 6.5 100 2 0 xprice`
`xyield`	`settle maturity cpn price redeem freq basis -- yld`	annual yield in percent of a dated bond from its clean price	`20080215 20161115 5.75 95.04287 100 2 0 xyield`
`xaccr`	`settle maturity cpn freq basis -- accrued`	accrued interest per 100 face at settlement	`20080215 20171115 5.75 2 0 xaccr`
`yearfrac`	`d1 d2 basis -- f`	year fraction between two dates; basis 0 = 30/360 US, 1 = actual/actual, 2 = actual/360, 3 = actual/365, 4 = 30E/360	`"1.1.2007" "1.1.2008" 3 yearfrac`

Command	Stack	Description	Example
`date>ymd`	`date -- yyyymmdd`	a date string d.m.yyyy as the integer YYYYMMDD (for cash-flow matrices)	`"15.3.2026" date>ymd`
`ymd>date`	`yyyymmdd -- date`	an integer YYYYMMDD as the date string d.m.yyyy of the dates module	`20260315 ymd>date`

Module `complex` 1.0 — complex numbers: parts, argument, conjugate, construction

Complex

Command	Stack	Description	Example
`re`	`z -- Re(z)`	real part (a real is unchanged)	`(1,2) re`
`im`	`z -- Im(z)`	imaginary part (0 for a real)	`(1,2) im`
`arg`	`z -- arg(z)`	argument (phase) in radians, in (-pi, pi]	`(0,1) arg`
`conj`	`z -- conj(z)`	complex conjugate (elementwise for matrices; identity for reals)	`(1,2) conj`
`re2c`	`x -- (x,0)`	promote a real to complex	`5 re2c`
`split_c`	`z -- Re(z) Im(z)`	split a complex number into real and imaginary parts	`(1,2) split_c`
`j2r`	`re im -- (re,im)`	join two reals into a complex number (imaginary part on top)	`1 2 j2r`

Module `special` 1.0 — gamma, beta and the standard normal distribution

Special functions

Command	Stack	Description	Example
`npdf`	`x -- phi(x)`	standard normal density	`0 npdf`
`ncdf`	`x -- Phi(x)`	standard normal cumulative distribution	`1.96 ncdf`
`nquant`	`p -- x`	standard normal quantile (inverse of ncdf), $0 < p < 1$	`0.975 nquant`
`gamma`	`x -- Gamma(x)`	gamma function	`5 gamma`
`ln_gamma`	`x -- ln Γ(x)`	log gamma function	`10 ln_gamma`
`beta`	`a b -- B(a,b)`	beta function	`2 3 beta`
`ln_beta`	`a b -- ln B(a,b)`	log beta function	`2 3 ln_beta`

Module `matrices` 1.0 — linear algebra, matrix construction and reduction, polynomials

Matrices

Command	Stack	Description	Example
`tran`	`A -- A^T`	transpose	`[2 3 $ 1 2 3 4 5 6] tran`

Command	Stack	Description	Example
'	A -- A'	transpose (alias of tran)	[2 3 $ 1 2 3 4 5 6] '
minv	A -- A^-1	inverse of a square real or complex matrix (error if singular)	[2 2 $ 4 7 2 6] minv
pinv	A -- A^+	Moore-Penrose pseudoinverse of a square real matrix (SVD)	[2 2 $ 1 2 2 4] pinv
det	A -- det(A)	determinant of a square real or complex matrix	[2 2 $ 4 7 2 6] det
eig	A -- V L	eigenvectors (columns of V) and eigenvalues (diagonal of L) of a square real matrix; both complex	[2 2 $ 2 1 1 2] eig
chol	A -- L	Cholesky factor of a symmetric positive definite real matrix, A = L L^T	[2 2 $ 4 2 2 3] chol
svd	A -- U S V	singular value decomposition of a real matrix, A = U S V^T	[2 2 $ 4 7 2 6] svd
kron	A B -- A(x)B	Kronecker product	[2 2 $ 1 2 3 4] [1 2 $ 5 6] kron
dim	A -- A rows cols	push the dimensions (the matrix stays)	[2 3 $ 1 2 3 4 5 6] dim
reshape	A rows cols -- B	reshape in row-major order; rows*cols must match	[2 3 $ 1 2 3 4 5 6] 3 2 reshape
get_aij	A i j -- A a_ij	push element (i,j), 0-based (the matrix stays)	[2 2 $ 1 2 3 4] 0 1 get_aij
set_aij	A i j x -- A	set element (i,j), 0-based, in place; a complex x promotes a real matrix	[2 2 $ 1 2 3 4] 0 1 9 set_aij
split_mat	A -- a11 a12 ...	push all elements as scalars, row by row	[2 2 $ 1 2 3 4] split_mat
diag	A -- v	main diagonal as a row vector	[2 2 $ 1 2 3 4] diag
to_diag	v -- D	diagonal matrix from a row or column vector	[1 3 $ 1 2 3] to_diag
eye	n -- I	n x n identity matrix	3 eye
ones	rows cols -- A	matrix filled with ones	2 3 ones
zeroes	rows cols -- A	matrix filled with zeros	2 3 zeroes
rand	rows cols -- A	matrix of uniform(0,1) randoms	2 2 rand
randn	rows cols -- A	matrix of standard normal randoms	2 2 randn
rrange	start step end -- v	row vector start, start+step, ... up to end	0 0.25 1 rrange
join_v	A B -- [A; B]	stack two matrices vertically (same number of columns)	[1 2 $ 1 2] [1 2 $ 3 4] join_v
join_h	A B -- [A B]	join two matrices side by side (same number of rows)	[2 1 $ 1 2] [2 1 $ 3 4] join_h
cumsum_r	A -- B	cumulative sum along each row	[2 2 $ 1 2 3 4] cumsum_r
cumsum_c	A -- B	cumulative sum down each column	[2 2 $ 1 2 3 4] cumsum_c
cmean	A -- v	mean of each column (row vector)	[2 2 $ 1 2 3 4] cmean
rmean	A -- v	mean of each row (column vector)	[2 2 $ 1 2 3 4] rmean
csum	A -- v	sum of each column (row vector)	[2 2 $ 1 2 3 4] csum
rsum	A -- v	sum of each row (column vector)	[2 2 $ 1 2 3 4] rsum

Command	Stack	Description	Example
`cvar`	`A -- v`	sample variance of each column (row vector)	`[2 2 $ 1 2 3 4] cvar`
`rvar`	`A -- v`	sample variance of each row (column vector)	`[2 2 $ 1 2 3 4] rvar`
`cmin`	`A -- v`	minimum of each column	`[2 2 $ 1 2 3 4] cmin`
`rmin`	`A -- v`	minimum of each row	`[2 2 $ 1 2 3 4] rmin`
`cmax`	`A -- v`	maximum of each column	`[2 2 $ 1 2 3 4] cmax`
`rmax`	`A -- v`	maximum of each row	`[2 2 $ 1 2 3 4] rmax`

Polynomials

Command	Stack	Description	Example
`roots`	`coeffs -- roots`	roots of a real polynomial; coefficients highest degree first, result is a complex row vector	`[1 3 $ 1 -3 2] roots`
`pval`	`coeffs x -- y`	evaluate a polynomial (coefficients highest degree first, real or complex) at x	`[1 3 $ 1 0 -1] 2 pval`

Module `numerics` 1.0 — integration, roots, minima, derivatives and sums over any word

Numerics

Command	Stack	Description	Example
`exec`	`xt --`	run an execution token (@name pushes one)	`3 @sq exec`
`integrate`	`a b xt -- I`	integral of f over [a,b] (tanh-sinh with adaptive bisection; handles endpoint singularities)	`0 pi @sin integrate`
`romberg`	`a b xt -- I`	integral of f over [a,b] by Romberg extrapolation	`0 1 @exp romberg`
`fzero`	`a b xt -- x`	root of f in the bracket [a,b] where f changes sign (Brent)	`2 4 @sin fzero`
`solve`	`guess xt -- x`	root of f near one guess (bracket search, then Brent)	`3 @sin solve`
`minimize`	`a b xt -- x`	argument of the minimum of f in [a,b] (Brent)	`0 5 @par minimize`
`deriv`	`x xt -- f'(x)`	derivative of f at x (Ridders extrapolation)	`2 @exp deriv`
`fsum`	`lo hi xt -- S`	sum of f(i) for integer i from lo to hi	`1 1000 @invsq fsum`
`fprod`	`lo hi xt -- P`	product of f(i) for integer i from lo to hi	`1 10 @ident fprod`
`tol`	`x --`	set the convergence tolerance (default 1e-10)	`1e-6 tol`
`nevals`	`-- n`	function evaluations used by the last numerical call	`nevals`
`interr`	`-- e`	error estimate of the last integrate	`interr`

Module `strings` 1.0 — string construction and manipulation

Strings

Command	Stack	Description	Example
`scon`	`s1 s2 -- s1s2`	concatenate two strings	`"foo" "bar" scon`
`substr`	`s start end -- sub`	substring from index start (0-based) up to end (exclusive)	`"abcdef" 1 3 substr`
`s2l`	`s -- s_lower`	convert string to lowercase	`"Hello" s2l`
`s2u`	`s -- s_upper`	convert string to uppercase	`"Hello" s2u`
`slen`	`s -- n`	length of a string in bytes	`"hello" slen`
`srev`	`s -- s_rev`	reverse string	`"abc" srev`
`int2str`	`x -- s`	integer part of x as a string	`123 int2str`

Module `registers` 1.0 — 64 storage registers of any type

Registers

Command	Stack	Description	Example
`sto`	`x n --`	store x in register n (0-63); any type	`42 0 sto`
`rcl`	`n -- x`	push a copy of register n	`0 rcl`
`ffr`	`-- n`	index of the first free register	`ffr`
`pr`	`--`	show which registers are occupied	`pr`
`saveregs`	`--`	save all registers to registers.txt in the config directory	`saveregs`
`loadregs`	`--`	load the registers saved by saveregs	`loadregs`
`clregs`	`--`	clear all registers	`clregs`

Module `dates` 1.1 — calendar arithmetic, workdays and holidays, time zones, sunrise and sunset

Dates

Command	Stack	Description	Example
`today`	`-- date`	today's date as a string d.m.yyyy	`today`
`now`	`-- "HH:MM"`	push the current local time as a string, same form as sunrise/sunset	`now`
`tz_offset`	`date zone -- hours`	UTC offset in hours of an IANA zone on the given date, DST included	`"1.12.2026" "America/New_York" tz_offset`
`ddays`	`d1 d2 -- n`	days from date d1 to date d2 (d2 - d1)	`"1.1.2025" "31.12.2025" ddays`
`dateplus`	`date n -- date2`	the date n days later (n may be negative)	`"1.1.2025" 30 dateplus`
`dow`	`date -- name`	weekday name of a date	`"1.1.2025" dow`
`edmy`	`date -- y m d`	split a date into year, month, day (day on top)	`"1.1.2025" edmy`

Command	Stack	Description	Example
num2date	d m y -- date	build a date string from day, month, year	1 1 2025 num2date
days2eoy	date -- n	days from the date to the end of its year	"1.1.2025" days2eoy
sunrise	date lat lon zone -- time	local sunrise HH:MM; zone is an IANA name or a UTC offset in hours	"21.6.2025" 38.9 -77.0 "America/New_York" sunrise
sunset	date lat lon zone -- time	local sunset HH:MM	"21.6.2025" 38.9 -77.0 -4 sunset
dawn	date lat lon zone -- time	civil dawn HH:MM (sun 6 degrees below the horizon)	today 38.9 -77.0 "America/New_York" dawn
dusk	date lat lon zone -- time	civil dusk HH:MM	today 38.9 -77.0 "America/New_York" dusk

Workdays

Command	Stack	Description	Example
workdays	d1 d2 -- n	workdays from d1 to d2, both inclusive (Excel NETWORKDAYS): Mondays to Fridays not in the holiday calendar; negative if d2 < d1	"1.1.2026" "31.12.2026" workdays
workdayplus	date n -- date2	the date n workdays after date (n may be negative; the start date is not counted, as Excel WORKDAY)	"24.12.2026" 10 workdayplus
isworkday	date -- flag	1 if the date is a Monday to Friday and not a holiday	20261126 isworkday
holidays	name -- H	the holiday calendar as a vector of YYYYMMDD: the file holidays/name.txt in the config directory, or the built-in rules "us" (federal, observed) and "si" (Slovenia) for the years around today	"us" holidays
yholidays	year name -- H	the built-in rules for one year	2027 "si" yholidays
sethol	H --	install a vector of dates YYYYMMDD as the holiday calendar for this session	[2 1 $ 20261126 20261225] sethol
usehol	name --	install the calendar ‘name‘ (file or rules) and remember it in config.txt	"us" usehol
clrhol	--	remove the holiday calendar (weekends only) and forget it in config.txt	clrhol
hol	-- H	the installed holiday calendar as a vector	hol

Module `programs` 1.0 — HP-41 style programs with labels, jumps and subroutines

Programs

Command	Stack	Description	Example
`run`	`s --`	load and run the HP-41 style program in file s (LBL/GOTO/GOSUB/RTN/END)	`"prog1.prg" run`

B The std word module

The following words are loaded from `modules/std.mm` in the configuration directory. The file is replaced at each `make install`; add your own words to `user_words.txt` or to a word module of your own.

Word	Definition	Meaning
`sq`	`dup *`	x^2
`qu`	`sq sq`	x^4
`cube`	`3 ^`	x^3
`cubrt`	`3 inv ^`	cube root (complex for negative x)
`pctchg`	`over - swap / 100 *`	percent change from y to x, consuming both
`pctot`	`/ 100 *`	x as a percentage of y, consuming both
`ln2`	`ln 2 ln /`	base-2 logarithm
`norm2`	`svd drop swap drop diag rmax split_mat`	spectral norm (largest singular value)
`normfro`	`dim * 1 reshape 2 .^ csum split_mat sqrt`	Frobenius norm
`cond`	`svd drop swap drop diag dup rmax swap rmin ./ split_mat`	2-norm condition number
`isnan`, `isinf`	`nan eq`, `inf eq`	flags
`isreal`	`im 0 eq`	1 if the imaginary part is zero
`selreal`	`dup isreal swap re .*`	keep the real entries of a vector, zero the rest
`cumprod_c`, `cumprod_r`	`ln cumsum_c exp`	cumulative products
`sto_add`, `sto_sub`, `sto_mul`, `sto_div`	`dup rcl 2 roll + swap sto` etc.	storage arithmetic: `x n sto_add` adds x to register n
`XprimeX`	`dup ' swap *`	$X^T X$
`f2c`, `c2f`	`32 - 9 / 5 *`, `9 * 5 / 32 +`	Fahrenheit and Celsius
`dec_hr`	`dup 0 2 substr eval swap 3 5 substr eval 60 / +`	`"HH:MM"` to decimal hours
`alexandria`, `duluth`, `durham`, `kranj`	`lat lon "zone"`	locations for the solar functions

C Coming from MM-15, MM-14, MM-11 or MM-B1

MM-41 1.0 was MM-15 2.1 restructured into modules; 1.1 added MM-B1's decimal numbers and finance words as the `decimal` and `finance` modules; 1.2 added workdays and holiday calendars to the dates module. Every MM-15 command, word and program runs unchanged, with one exception noted below. What is different:

- The binary is `mm41`, the configuration directory `~/.config/mm41/`. Copy `user_words.txt`, `registers.txt` and `config.txt` across from `mm_15` if you want them.
- `predefined_macros.txt` is now the word module `modules/std.mm` and is installed there; the old file is not read. Your own additions belong in `user_words.txt` or in a module of your own.
- New commands: **`modules`**, **`use`**, **`unuse`**, **`load`**; the option `--modules`; `make MODULES=...` and `make shared-module`.
- **`listfcns`**, `"name" usage` and the generated reference name the module a command comes from.
- The `std` macros `npv` and `irr` (latest flow first, rate as a fraction) are replaced by the finance module's **`npv`** and **`irr`** (time order, rate in percent); see Section 13.3.

MM-11 and MM-14 users: MM-15 2.0 already contained both (the same core, words, control flow and numerical methods), so MM-41 is the one calculator to install.

MM-B1 users: every MM-B1 word is here with the same name and argument order, except that

- cash flows are matrices on the stack rather than a bank filled with `cf`/`cfn`/`xcf`: `-1000 cf 300 cf 5 cfn` becomes `[2 2 $ -1000 1 300 5]`, and `xcf` rows become an $N \times 2$ matrix of amount and date (Sections 13.3 and 13.6); `cfs`, `clrcf`, `xcfs` and `clrxcf` are not needed;
- `i` is spelled **`i%`** and `?i` is **`?i%`**; `begin` is **`beg`** (`begin` is a control word);
- `fix` is **`setprec`**; `date+` is **`dateplus`** and `ddays` is in the dates module, both of which also accept `YYYYMMDD` numbers through **`ymd>date`**; `dow` returns the weekday name;
- numbers are binary by default; **`decimal`** switches the number mode, and the finance words return decimals whatever the mode.

`CHANGES.md` in the source tree has the full history, including the MM-15 2.x changes from 1.x.